EE5203 L01 Study Sheet

Embedded Systems, the C Toolchain, and Debugging - Basri Erdogan

Essential question: How does a line of C code become an observable change on an STM32 board, and what evidence verifies each step?

What you should be able to do

- Distinguish a processor, microcontroller, and embedded system.
- Trace source code through compile, link, flash, and execution.
- Identify the compiler, linker, programmer, and debugger roles.
- Explain why a successful build, successful flash, and correct behavior are different claims.
- Use a breakpoint and variable inspection as engineering evidence.
- Describe the course policy: **register literacy, HAL fluency**.

Core model

```
Input -> STM32 microcontroller -> Output
              |
       software + memory
       + peripherals + timing
```

Term	Meaning in this course
Processor	The CPU core that executes instructions
Microcontroller	Processor, memory, and peripherals on one chip
Embedded system	Purpose-built hardware and software performing a dedicated function
Peripheral	Hardware block such as GPIO, timer, ADC, UART, I2C, or SPI

Toolchain path

```
C source -> compiler -> object files -> linker -> .elf firmware
                                                      |
                                                flash/program
                                                      v
                                                 STM32 memory
```

Stage	Evidence	Typical failure
Compile	Source translated; errors/warnings reported	Syntax, type, or undeclared name
Link	Objects/libraries combined into `.elf`	Missing definition or memory problem
Flash	Firmware written to the board	Board/debug connection problem
Run/test	Observable requirement checked	Logic, configuration, or hardware problem

First C pattern

```c
while (1)
{
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    HAL_Delay(500);
}
```

- `while (1)` repeats indefinitely.
- `HAL_GPIO_TogglePin(...)` changes the physical LD2 state.
- `HAL_Delay(500)` blocks for approximately 500 ms.
- We use HAL here to learn the toolchain. Direct GPIO registers follow after the four-lecture Embedded C Foundation.

Board facts

Item	Nucleo-F401RE fact
MCU	STM32F401RE
Processor core	Arm Cortex-M4
Programmer/debugger	On-board ST-LINK
User LED	LD2 on PA5
User button	B1 on PC13
Development environment	VS Code + STM32CubeMX (STM32CubeCLT toolchain)

Debugging pattern

```
Predict -> Run -> Observe -> Compare -> Change one thing -> Repeat
```

Before asking for implementation help, state:

1. expected behavior;
2. observed behavior;
3. evidence collected;
4. next smallest test.

Common mistakes

- Treating "build succeeded" as proof that the hardware works.
- Reading the last compiler error instead of the first relevant error.
- Editing one project but building/flashing a different project.
- Writing application code outside protected `USER CODE` regions.
- Changing several settings at once and losing the cause-effect relationship.
- Using a power-only USB cable rather than a data cable.

Mastery self-check

☐ I can explain compiler versus linker.
☐ I can identify the `.elf` file's role.
☐ I can distinguish build, flash, and behavior evidence.
☐ I can create a breakpoint and predict where execution pauses.
☐ I can explain why `HAL_Delay(100)` blinks faster than `HAL_Delay(500)`.
☐ I can name the board, MCU, core, programmer, LED pin, and button pin.
☐ I can describe when this course uses registers and when it uses HAL.

Five review questions

1. What does a microcontroller contain that a processor alone may not?
2. What is the linker's job?
3. What does a successful flash prove?
4. Why is a breakpoint useful in a working program?
5. What single observation would confirm that a delay modification reached the board?

See the L01 worksheet for extended practice. Worked solutions are released after the practice window.